DriveWAM 기반 Lightweight Action Readout 연구

송용휘

요약

DriveWAM은 5B 규모의 video diffusion transformer로 미래 영상을 먼저 상상하고, 같은 backbone을 10회 더 실행해 주행 action을 생성한다. 이 연구는 backbone을 그대로 고정한 채, **미래를 한 번 상상한 시점의 중간 layer 표현**을 약 1,000만 파라미터의 작은 head(SF3)로 읽어 4초 궤적을 직접 회귀한다. 제안 모델은 첫 상상 pass 직후 추론을 끝내므로, 나머지 미래 denoising과 action denoising 10회를 모두 건너뛴다. NAVSIM(장면 단위 계획)과 PhysicalAI-AV(연속 주행) 두 벤치마크에서 사전등록한 protocol로 평가했다.

표 1. 핵심 결과

항목	NAVSIM (잠금 navtest-final, 8,896장면)	PhysicalAI-AV (공식 test 1k)
공식 DriveWAM (B10)	PDMS 89.67 (p50 797.9 ms)	ADE 0.802 m / FDE 2.384 m (p50 837.8 ms)
B* (품질을 지키는 가장 빠른 저-step 기준)	B02-opt, action 2 step (p50 420.7 ms)	B03-opt, action 3 step (p50 434.0 ms)
사전에 정한 품질 기준 (B10·B* 대비)	PDMS 차이의 95% CI 하한 ≥ –2.0, NC·DAC 평균 악화 ≤ 0.5, p50 ≤ 0.8 × B*, seed 2개 모두 충족	ΔADE CI 상한 ≤ +0.05 m, ΔFDE CI 상한 ≤ +0.15 m, FDE > 2 m 비율 증가 ≤ 1%p, p50 ≤ 0.8 × B*, p95 악화 없음, seed 3개 모두 충족
제안 모델 (SF3-first) 품질	PDMS 89.25 / 89.49 (seed 2개)	ADE 0.750 / 0.748 / 0.762 m, FDE 2.244 / 2.244 / 2.287 m (seed 3개)
제안 모델 결정 지연 p50	189.0 ms: B*보다 2.2배, B10보다 4.2배 빠름	70.9 ms: B*보다 6.1배, B10보다 11.8배 빠름
사전등록 판정	근소 미통과 (seed 17의 DAC 조건, 한 장면 차이)	통과 (seed 3개 × 기준 7개)

핵심 발견

- **상상 한 번이 중요하다.** 미래 latent를 한 번 처리한 first 표현이 관측만 처리한 current 표현보다 훨씬 유용했다(NAVSIM +3.3 PDMS, PhysicalAI ADE –0.11 m). 미래 생성을 끝까지 마친 terminal 표현은 first보다 나아지지 않았다.
- **속도.** 반복 action 생성을 통째로 없앤 제안 모델은, action 단계 수만 줄인 가장 빠른 기준 B*보다도 NAVSIM에서 2.2배, PhysicalAI에서 6.1배 빠르다. 공식 DriveWAM(B10)과 비교하면 각각 4.2배, 11.8배 빠르다.
- **데이터 규모.** head 학습 데이터를 4k → 20k → 92.6k 장면으로 늘릴수록 B10과의 격차가 줄었다(NAVSIM PDMS –4.2 → –1.3 → –0.4). PhysicalAI도 4k에서는 미달, 20k에서 통과했다.
- **남은 약점.** NAVSIM에서는 주행 가능 영역 준수(DAC), PhysicalAI에서는 4초 끝점 오차(FDE)가 남아 있다. 두 약점의 원인은 아직 검증하지 않았다. PhysicalAI에서 제안 모델이 B10보다 낮은 오차를 낸 것은 상당 부분 결정적(평균) 예측 효과이므로 정확도 우위로 해석하지 않는다.

1. 배경: DriveWAM의 구조와 병목

1.1 World-Action Model

일반적인 end-to-end 주행 정책은 현재 관측에서 바로 궤적을 출력한다. World-Action Model(WAM)은 먼저 미래 환경이 어떻게 변할지를 표현(상상)하고, 그 미래와 일치하는 action을 생성한다. 주변 차량과 보행자, 신호, ego 차량 상태가 계속 바뀌는 자율주행에서는 미래 dynamics를 명시적으로 다루는 이런 구조가 자연스럽다. DriveWAM은 대규모 video generative model이 이미 학습한 세계의 동역학을 주행 정책의 backbone으로 쓰는 대표적인 사례다.

1.2 DriveWAM의 추론 흐름

DriveWAM은 사전학습된 video diffusion transformer(Wan2.2-TI2V-5B, 30 layer; 전체 6.24B 파라미터)를 정책의 중심 backbone으로 쓴다. 구성 요소는 Video VAE, Action Encoder(MLP), Ego-state Encoder, Video Diffusion Transformer, Scene-Evolving VLM Guidance(Qwen3-VL-8B), Action Decoder(MLP), Selective KV Memory다. 한 chunk(4초)에 대한 추론은 다음 순서로 진행된다.

1. 현재·과거 영상을 VAE로 latent로 바꾸고, 과거 action(MLP encoder), ego state, VLM 안내 문장과 함께 history로 등록한다.
2. 미래 영상 latent를 flow matching으로 3 step 생성한다(미래 상상).
3. 생성한 미래 latent를 조건으로 **같은 5B transformer를 10 step 더 실행**해 action 표현을 만들고, MLP decoder로 4초 궤적으로 바꾼다. 논문은 이 과정을 "예측한 미래에 근거한 inverse-dynamics readout"으로 설명한다.

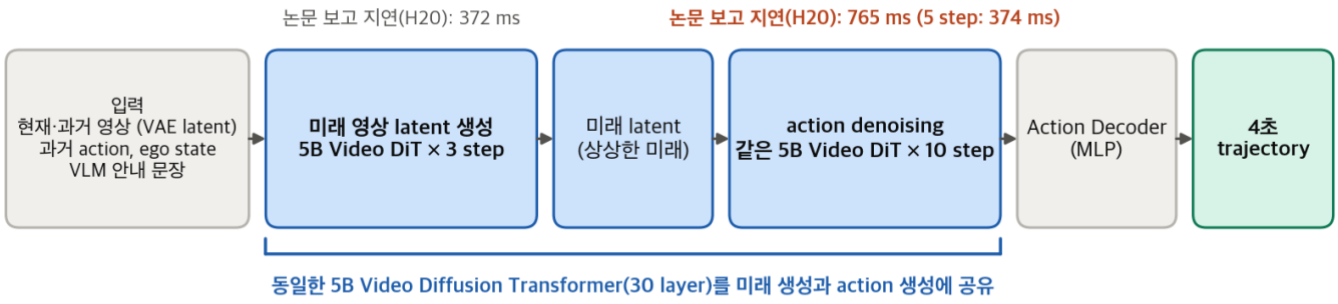


그림 1. DriveWAM의 한 chunk 추론 흐름. 미래 생성과 action 생성이 같은 5B backbone을 쓴다.

VLM 안내는 최신 관측, 최근 ego 궤적, route command를 입력받아 "Proceed straight with caution, slowing near pedestrians..." 같은 문장을 만들고, chunk마다 한 번 생성해 여러 denoising step에서 재사용한다. Selective KV Memory는 긴 주행에서 history가 쌓여 KV cache가 커지는 문제를 token의 relevance와 redundancy 점수로 관리한다. 두 요소 모두 이 연구에서는 손대지 않고 그대로 쓴다.

1.3 Action 생성이 비싼 이유

Action 출력은 4초 궤적을 이루는 수십 개의 숫자뿐이다. 그런데 이것을 만들기 위해 5B backbone을 10회 반복 실행한다. 논문이 보고한 지연 분해(H20 GPU 1장, 4초 chunk)는 VLM 안내 문장 생성 125 ms, 미래 영상 latent

생성(3 step) 372 ms, action denoising 10 step 765 ms(5 step이면 374 ms)로, action denoising이 가장 큰 단일 병목이었다.

논문은 action step을 10에서 5로 줄였을 때 ADE 0.83 → 0.84 m, FDE 2.47 → 2.45 m로 품질이 거의 변하지 않았다고 보고한다. action 쪽 계산에 상당한 중복이 있다는 단서다. 따라서 병목은 "action decoder가 크다"가 아니라, **이미 미래 표현을 만들었는데도 action 표현을 얻기 위해 같은 backbone을 여러 번 다시 실행한다**는 구조 자체에 있다.

1.4 연구 질문

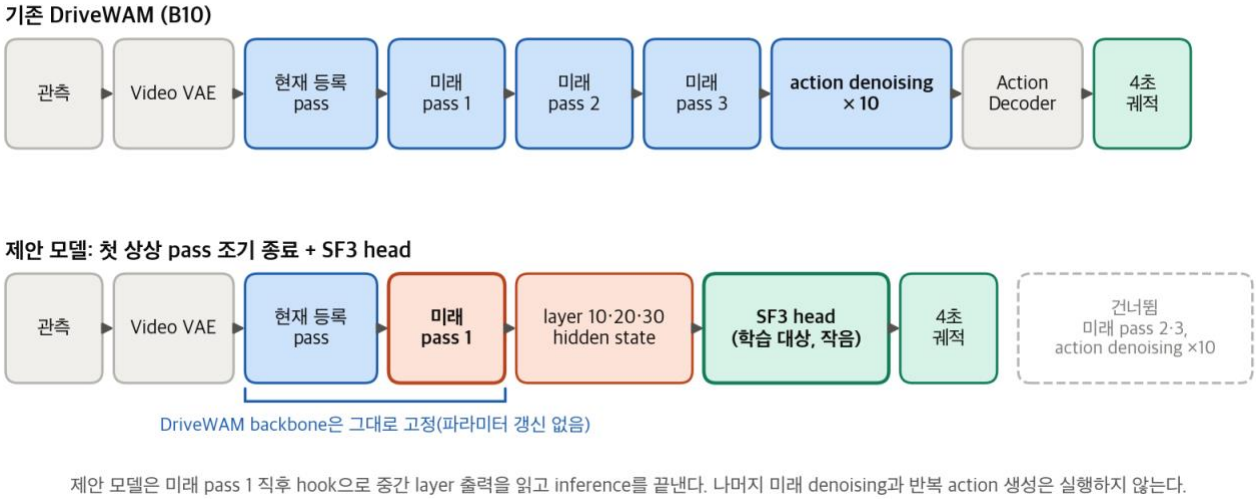
이 연구의 질문은 다음과 같다. 이미 학습된 DriveWAM이 미래를 상상하는 과정에서 만든 내부 표현만으로, 반복 action denoising 없이 작은 head가 궤적을 읽어낼 수 있는가. 그렇다면 어느 시점과 깊이의 표현이 필요하고, 속도와 품질은 어떻게 되는가.

backbone은 재학습하지 않고 head만 학습하므로, 이 연구는 "DriveWAM에 새로운 주행 능력을 가르치는" 연구가 아니라 "이미 내부에 있는 표현에서 action을 얼마나 가볍게 읽어낼 수 있는가"를 묻는 post-hoc readout 연구다. 구조적 아이디어는 Light-WAM(현재 관측 표현에서 action을 직접 decoding, state fusion)과 Faster-WAM(얕은 action head에 여러 깊이의 world 표현 전달)에서 가져왔다. 차이는 ① 완전히 고정된 주행 checkpoint 위에서 자기 자신의 action 생성기를 대체한다는 점, ② 표현을 읽는 시점(current/first/terminal)을 체계적으로 비교한다는 점, ③ 단계 수만 줄인 강한 기준(B*)과 사전등록 protocol로 속도·품질을 비교한다는 점이다.

2. 제안 방법: 첫 상상 pass의 표현을 읽는 가벼운 head

2.1 전체 구조

그림 2는 기존 DriveWAM과 제안 구조를 같은 축에서 비교한다. 제안 구조는 관측 등록과 첫 번째 미래 pass까지는 원본과 완전히 같고, 그 직후 중간 layer의 hidden state를 읽어 작은 head로 궤적을 회귀한 뒤 추론을 끝낸다. 원본이 이어서 수행하는 미래 pass 2·3, terminal pass, action denoising 10회, 원래 action decoder는 모두 실행하지 않는다. 계산 절감은 전부 이 조기 종료에서 나온다.



제안 모델은 미래 pass 1 직후 hook으로 중간 layer 출력을 읽고 inference를 끝낸다. 나머지 미래 denoising과 반복 action 생성은 실행하지 않는다.

그림 2. 기존 DriveWAM(B10)과 제안 구조(SF3-first)의 추론 경로 비교.

2.2 고정 backbone과 표현 추출

NAVSIM과 PhysicalAI 각각의 공개 DriveWAM checkpoint를 그대로 쓴다. 공식 추론 코드를 비트 단위로 재현한 rollout에 forward hook을 달아 transformer block의 출력을 읽으며, hook은 어떤 출력도 바꾸지 않는다. 구체적인 설정은 다음과 같다.

- **읽는 layer:** 30개 block 중 10, 20, 30번째(0-based index 9, 19, 29). 각 layer 출력은 영상 token 112개 × 3,072차원(bf16)이며, 두 벤치마크에서 같은 크기다.
- **조기 종료:** 제안 모델은 첫 미래 pass의 출력이 모이면 예외로 rollout을 끊고 KV cache를 비운다. 조기 종료로 얻은 feature와 전체 추론을 끝까지 돌렸을 때 같은 지점의 feature가 비트 단위로 같음을 두 벤치마크 모두에서 확인했다.
- **학습 범위:** backbone 파라미터는 갱신하지 않는다. 학습되는 것은 새로 붙인 head뿐이다. feature는 offline으로 저장해 head를 학습하고, 최종 평가는 실제 추론 중 feature를 읽는 online 경로로 수행했다(두 경로의 출력이 같음을 확인).

2.3 미래 pass의 내부 동작

backbone의 30개 block은 각각 self-attention, 안내 문장에 대한 cross-attention, ego 조건에 대한 cross-attention, feed-forward로 이루어지며, timestep 조건은 scale·shift·gate 변조로 들어간다. self-attention에는 KV cache가 붙어 있다. 등록 pass에서는 관측 latent token(그리고 과거 action token)이 각 layer를 지나며 만든 key와 value가 cache에 기록된다. 이것이 모델이 기억하는 현재 장면과 history다.

미래 pass는 미래 프레임 자리에 순수 noise token 112개를 넣어 실행한다. 각 layer에서 noise token의 query가 cache에 쌓인 관측·history의 key·value를 self-attention으로 읽고, 안내 문장과 ego 조건을 cross-attention으로 읽은 뒤 feed-forward를 거친다. 이 과정이 30 layer 반복되면서 noise token의 hidden state는 지금 장면이 앞으로 어떻게 변할지를 담은 표현으로 바뀌고, 마지막에 flow 예측으로 변환되어 latent 갱신에 쓰인다. 미래 pass 중에는 noise token 자신의 key·value를 cache에 남기지 않고, 갱신이 끝난 마지막 pass에서만 미래 token의 key·value를 cache에 기록해 다음 chunk의 history로 넘긴다. 제안 모델이 읽는 표현은 첫 미래 pass에서 layer 10·20·30을 통과한 이 noise token의 hidden state다. 입력 자체는 noise이므로, 이 표현에 담긴 장면 정보는 전부 attention을 통해 cache와 조건에서 끌어온 것이다.

2.4 표현을 읽는 시점: current / first / terminal

DriveWAM의 미래 생성은 한 번의 forward로 끝나지 않고 미래 latent를 여러 번 갱신한다. 그래서 같은 layer라도 **언제** 읽느냐에 따라 표현의 내용이 다르다. 이 연구는 세 시점을 같은 head 구조로 각각 학습해 비교한다(가설 H4).

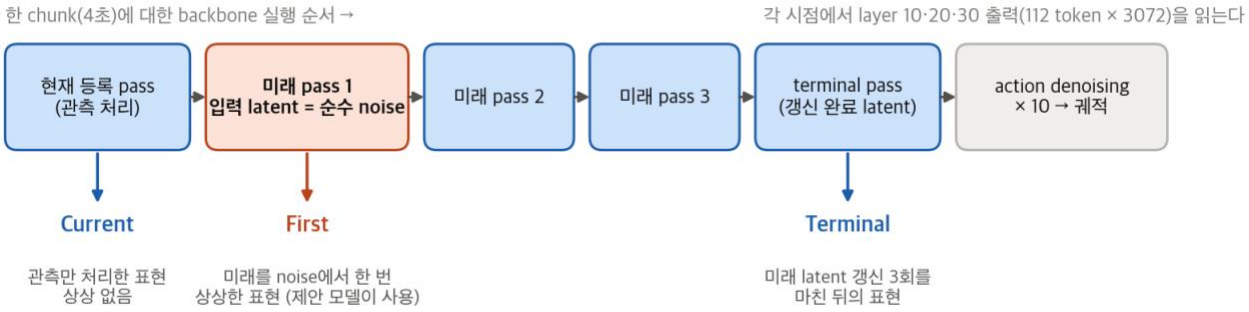


그림 3. 한 chunk 안에서 backbone이 실행되는 순서와 세 가지 표현 추출 시점.

- **Current:** 관측 프레임을 등록하는 pass의 표현. 미래 latent와 상호작용하기 전이다. "현재 장면 표현만으로 궤적을 읽을 수 있는가"의 기준점이 된다.
- **First:** 순수 noise 상태의 미래 latent를 입력해 처음 실행한 미래 pass의 표현. 미래를 noise에서 한 번 상상한 상태로, 아직 solver 갱신은 없다. 제안 모델이 최종적으로 쓰는 표현이다.
- **Terminal:** 미래 latent 갱신 3회를 마친 뒤 마지막 pass의 표현. 원본의 미래 생성을 끝까지 수행해야 하므로 first보다 비싸다.

first와 terminal의 비교가 핵심이다. terminal이 first보다 뚜렷이 좋다면 action을 읽기 위해 정제된 미래가 필요하다는 뜻이고, 두 결과가 비슷하다면 반복 refinement는 readout에 추가 정보를 거의 주지 않는다는 뜻이 된다. 후자라면 첫 pass에서 조기 종료하는 구조가 정당화된다.

2.5 SF3 head 구조

head는 세 layer의 token 표현과 ego state만 입력으로 받는다. 이름의 SF3는 State-Fusion, 3-level의 약자다. 코드(lightweight_heads.py, heads_v3.py, pai_heads.py) 기준 구조는 그림 4와 같다.

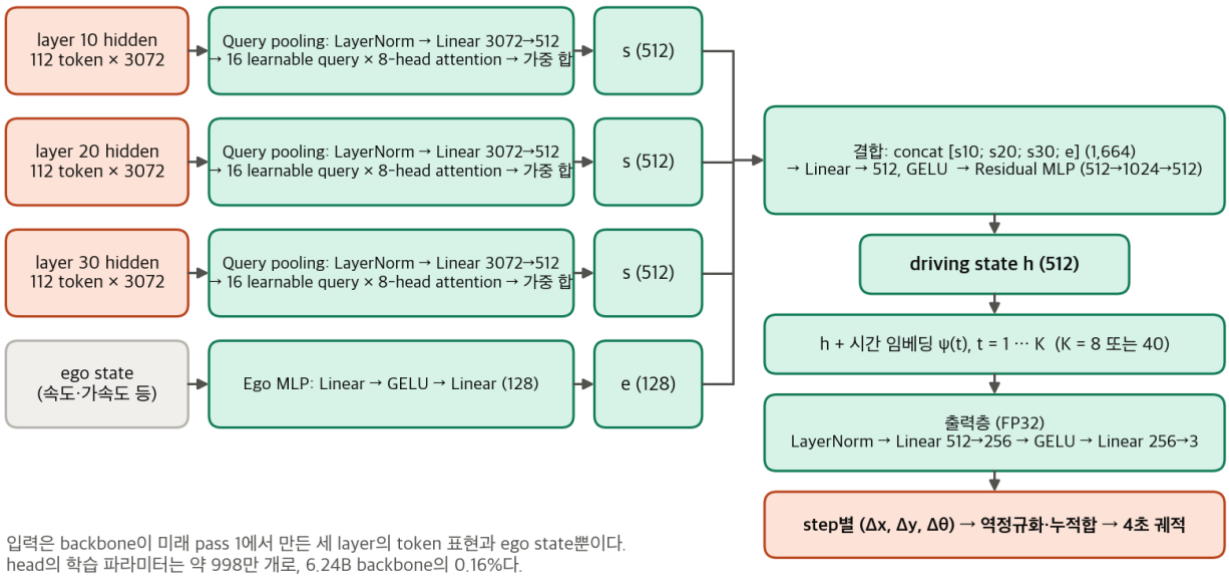


그림 4. SF3 head의 구조. 세 layer의 token 표현을 query pooling으로 압축하고 ego state와 결합한 뒤 시간 임베딩으로 K개 step을 출력한다.

표 2. SF3 head 설정 (코드 기준)

항목	설정
입력 layer	10, 20, 30 (0-based 9, 19, 29)
layer당 입력	112 token × 3,072
query pooling	LayerNorm → Linear 3,072→512 → 16 query × 8-head attention → 학습된 가중 합 → LayerNorm
ego 임베딩	Linear → GELU → Linear (128)
융합·trunk	concat(1,664) → Linear 512 + GELU → residual MLP 512→1,024→512
출력	h + ψ(t) → LayerNorm → Linear 512→256 → GELU → Linear 256→3, K = 8 (NAVSIM) / 40 (PhysicalAI)
학습 파라미터	약 998만 개 (backbone 6.24B의 0.16%)

- **Query pooling.** 각 layer의 112개 token을 모두 평균하면 도로·차량·하늘·배경 token이 같은 비중으로 섞인다. 대신 16개의 학습 가능한 query가 8-head attention으로 token을 읽고, 학습된 가중치로 합쳐 512차원 상태 하나로 만든다. "작은 query가 큰 world 표현에서 action에 필요한 정보만 골라 읽는" 역할이다.
- **여러 깊이의 결합.** 세 layer의 상태와 ego 임베딩(128)을 이어 붙여(1,664) 512차원으로 줄이고 residual MLP(512→1,024→512)를 거쳐 driving state h를 만든다. 특정 깊이 하나에 action 정보가 모두 있다고 가정하지 않기 위한 설계다(가설 H2).
- **Ego state 결합.** 같은 도로 장면이라도 속도가 5 km/h인지 70 km/h인지에 따라 궤적은 달라진다. world 표현이 ego 운동 상태를 모두 담는다고 보장할 수 없어 ego state를 별도 MLP로 넣는다(NAVSIM: 과거 4 pose·속도·가속도·command 20차원, PhysicalAI: 속도·가속도·곡률 5차원).
- **시간 임베딩과 출력.** 하나의 h에 step별 임베딩 ψ(t)를 더해 K개 시점을 구분한다(NAVSIM: 4초 2 Hz, K=8; PhysicalAI: 4초 10 Hz, K=40). 출력층(FP32)은 step별 정규화된 (Δx, Δy, Δθ)를 내고, 역정규화 후 누적합으로 궤적을 만든다. 정규화에는 학습 데이터 delta의 1%·99% 분위수를 쓴다.

2.6 학습

손실은 세 항의 합이다. 복원한 궤적 위치에 대한 Huber 손실, heading 차이에 대한 1 – cos(Δθ) 손실(가중치 0.2), 정규화된 delta에 대한 Huber 손실(가중치 0.1)이다. 학습 protocol은 초기 pilot에서 early stopping과 평가 세트가 섞여 쓰인 문제를 고친 뒤 v2부터 모든 실험에서 고정했다.

표 3. head 학습 protocol (v2 이후 고정)

항목	설정	항목	설정
epoch	40 고정	optimizer	AdamW, lr 3×10^{-4}, weight decay 0.01
batch	32 (학습) / 1 (예측, 배포와 동일)	schedule	warm-up 5% + cosine decay
checkpoint	마지막 epoch (평가로 선택하지 않음)	출력층 정밀도	FP32 (bf16에서는 batch 크기가 cm 단위로 결과를 바꿨음)

2.7 가설

표 4. 연구 가설

가설	질문	확인 방법
H1 표현의 충분성	고정된 backbone의 표현만으로 작은 head가 B10에 가까운 궤적을 낼 수 있는가	B10·B* 대비 품질 기준 (PDMS 또는 ADE/FDE)
H2 여러 깊이의 가치	layer 하나보다 세 깊이를 함께 읽는 것이 유리한가	SF3 vs 파라미터를 맞춘 단일 깊이 head (SF1-wide)
H3 실용 효율	품질을 지키는 가장 빠른 저-step 기준 B*보다 실제 지연이 낮은가	같은 세션 교차 측정 p50/p95 비율
H4 표현 시점	current / first / terminal 중 어느 시점이 readout에 가장 유용한가	같은 head를 세 시점으로 각각 학습해 비교

2.8 비교 기준과 B*

제안 모델이 원본 B10보다 빠르다는 것만으로는 부족하다. 새 head를 만들지 않고 기존 action denoising의 step 수만 줄여도 빨라지기 때문이다. 그래서 다음 기준을 함께 평가했다.

- **B10:** 공식 설정(미래 생성 3 step + action denoising 10 step). 품질과 지연의 기본 참조점이다.
- **opt 기준 B10/B05/B03/B02/B01-opt:** 출력에 영향이 없는 마지막 action cache pass를 생략하고 action step을 10/5/3/2/1로 줄인 변형. 생략 전후 출력이 비트 단위로 같음을 확인한 뒤 사용했다.
- **참고:** ego state만 쓰는 ego-only head, 등속(CV) 외삽.

B*는 개발 세트(dev)에서 B10 대비 사전에 정한 품질 기준을 만족하는 opt 기준 중 가장 빠른 것으로 정의한다. 제안 모델의 속도는 B10이 아니라 이 B*와 비교한다. 즉 질문은 B10보다 빠른가가 아니라, step을 최대한 줄인 가장 효율적인 기존 경로보다도 빠르면서 품질을 지키는가다.

3. 실험 설정

3.1 두 벤치마크

성격이 다른 두 벤치마크를 썼다. NAVSIM은 처음 보는 장면 하나에 대한 계획 품질을 규칙 기반으로 채점하고, PhysicalAI-AV는 시간적으로 이어진 clip에서 기억과 언어 안내가 있는 조건의 궤적 예측 정확도를 잰다. 각 환경의 공개 checkpoint와 공식 평가 방식을 그대로 따랐다.

표 5. 두 벤치마크의 설정

항목	NAVSIM v1.1	PhysicalAI-AV
backbone 입력	현재 전방 영상 1장, ego history(4 pose), 속도·가속도, 고정 route 문장	전방 카메라 clip(1 fps 21 frame), 과거 chunk 기억(KV), chunk별 VLM 안내 문장
평가 단위	장면(snapshot) 1개 → 4초 궤적 1개	clip 1개 → 연속 5 chunk × 4초 (teacher forcing)
head 출력	8 pose (2 Hz)	40 step × (Δx, Δy, Δθ) (10 Hz)
품질 지표	공식 PDMS와 하위 지표 NC, DAC, EP; ADE4 보조	공식 ADE / FDE; FDE > 2 m 비율 보조
head 학습 데이터	navtrain head-train 4k → 20k → 92.6k 장면	DriveWAM 공개 train 4k ⊂ 20k clip
dev / final	navtest-dev 34 로그 3,250장면 / 잠금 final 102 로그 8,896장면	val 1,000 clip (해시 순위) / 공식 test 1k
B* 자격 (B10 대비)	PDMS CI 하한 ≥ –2.0, NC·DAC 평균 악화 ≤ 0.5	ΔADE CI 상한 ≤ +0.05 m, ΔFDE CI 상한 ≤ +0.15 m
dev 결과 B*	B02-opt	B03-opt

3.2 NAVSIM 데이터와 분할

OpenScene revision a76f840b을 고정했다. navtrain(1,192 로그, 103,288 token)을 로그 단위로 나누어 head-train 92,599장면(1,072 로그)과 head-val 10,689장면을 만들었다. 초기 pilot은 head-val로 평가했는데, 분석 과정에서 공개 backbone이 navtrain 장면을 강하게 기억하고 있다는 사실이 드러났다(4.1절). 그래서 최종 protocol에서는 처음 보는 navtest를 로그 단위로 dev(34 로그, 3,250장면)와 잠금 final(102 로그, 8,896장면)로 나눴다. 로그 단위로 나눈 이유는 같은 주행 sequence의 비슷한 장면이 양쪽에 섞이는 것을 막기 위해서다. final은 설정을 모두 고정한 뒤 한 번만 평가했다.

3.3 PhysicalAI-AV 데이터와 평가 방식

NVIDIA dataset revision 33f9bf44를 고정하고 전방 카메라를 clip 단위로 처리했다(24,000 clip 중 23,994 처리, 실패 6개는 모두 val). head 학습에는 DriveWAM이 공개한 train 4k와 이를 포함하는 20k clip을 썼다. dev는 val 1,000 clip(해시 순위로 선정), final은 공식 test 1k다. 안내 문장은 DriveWAM 파이프라인대로 Qwen3-VL-8B로 생성했다(평균 27.8단어, final 28.0단어).

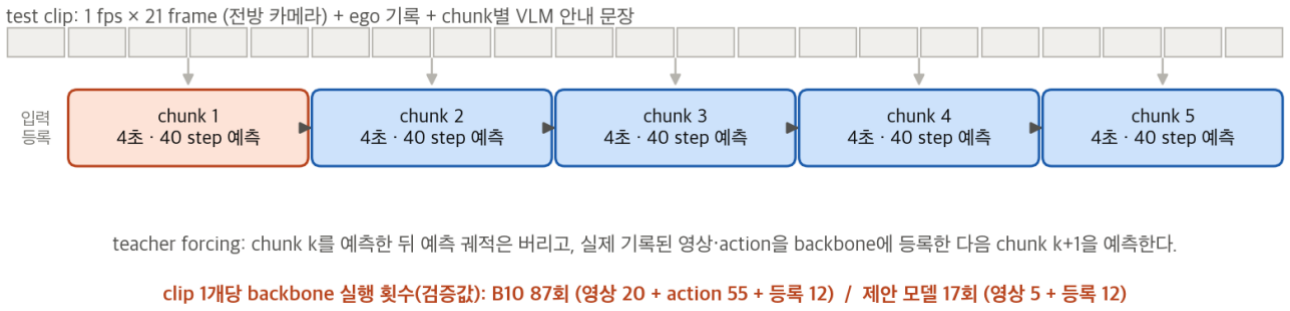


그림 5. PhysicalAI 평가 설정. clip 하나에서 5개 chunk를 teacher forcing으로 순서대로 예측한다.

평가는 공식 코드와 같은 teacher forcing이다. chunk k의 궤적을 예측한 뒤 그 결과를 다음 상태로 쓰지 않고, 실제 기록된 영상과 action을 등록하고 chunk k+1을 예측한다. 따라서 이 결과는 폐루프 주행 안전성이 아니라, 실제 주행 history가 주어진 조건에서의 미래 궤적 예측 능력으로 해석한다. 한 가지 좌표계 처리를 사전등록(변경 9)했다.

데이터의 action은 clip 시작 좌표계의 변위인데, head는 chunk가 시작하는 시점의 차량 좌표계에서 예측하고, 이미 등록된 실제 action으로 누적한 heading만큼 회전해 되돌린다. 미래 정보는 쓰지 않으며 모든 방법이 같은 기억을 받는다.

3.4 지표와 통계

NAVSIM은 공식 PDMS를 주 지표로, 충돌(NC), 주행 가능 영역 준수(DAC), 진행(EP), 4초 ADE를 함께 봤다. PhysicalAI는 공식 ADE(4초 평균 위치 오차)와 FDE(4초 끝점 오차)를 주 지표로, FDE가 2 m를 넘는 chunk 비율을 꼬리 오차 지표로 추가했다. 평균 차이만 보지 않고 paired bootstrap 2,000회로 신뢰구간을 계산했다. 장면을 독립 표본으로 다루지 않고 NAVSIM은 로그, PhysicalAI는 NVIDIA chunk 단위로 묶어 resampling했다.

3.5 지연 측정

모든 실험은 RTX A5000 24 GB 2장이 있는 공유 GPU 호스트에서 수행했다. 다른 사용자의 부하가 날마다 달라 절대 시간은 세션마다 차이가 났다(예: NAVSIM dev 세션은 load average 44, final 세션은 12). 그래서 같은 세션 안에서 장면 또는 clip마다 모델 실행 순서를 바꿔 번갈아 3회 측정하고, 판정은 B*에 대한 비율로 했다. 반복 간 p50 변동은 0.2~1.5%였다. 보고하는 값은 policy p50(NAVSIM, 관측 batch 구성부터 궤적 출력까지)과 결정 지연 p50/p95(PhysicalAI)다. VAE와 문장 인코더 비용은 모든 방법에 공통이라 비교에서 제외했다.

3.6 사전등록과 재현성 검증

결과를 본 뒤 성공 기준을 바꾸는 일을 막기 위해 데이터 분할, B* 선정 규칙, 제안 모델 사양, 지표, 판정 기준, 통계 절차, final 절차를 final 평가 전에 문서로 고정했다. 실행 중 바꾼 사항은 결과를 보기 전에 이유와 함께 기록했다. 전체 절차는 그림 6과 같다.

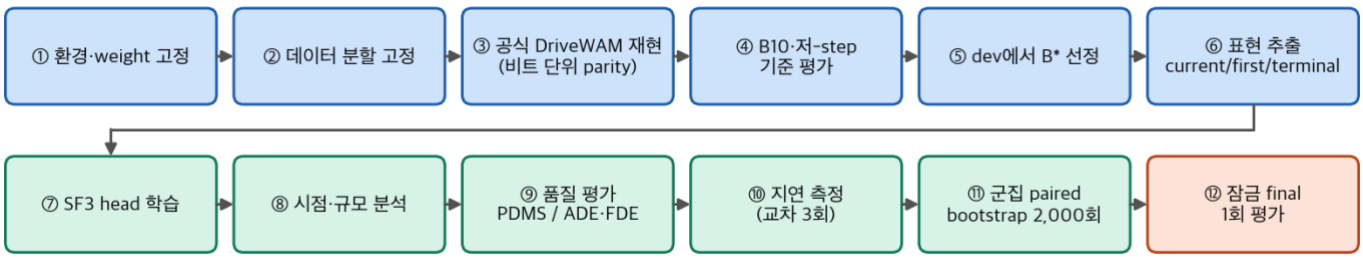


그림 6. 전체 평가 절차. final 세트는 한 번만 평가한다.

원본 코드에 hook, 조기 종료, opt 경로를 추가했으므로 코드 변경이 결과를 바꾸지 않았는지 다음 다섯 가지를 비트 단위로 확인했다. ① 수정 전 DriveWAM과 실험 코드의 rollout 출력 일치, ② opt 기준과 원본 출력 일치, ③ hook 추가 전후 backbone 출력 일치, ④ 조기 종료 feature와 전체 실행 feature 일치, ⑤ 저장한 feature로 평가한 offline 경로와 실제 추론 중 feature를 읽는 online 경로의 출력 일치. PhysicalAI에서는 clip당 backbone 실행 횟수도 함께 검증했다(B10 87회, 제안 모델 17회).

4. 결과

4.1 NAVSIM: backbone의 학습 장면 기억 효과

본 평가에 앞서 확인한 사실이다. 공개 backbone은 navtrain 장면을 강하게 기억하고 있어, navtrain 내부 검증 세트에서는 B10의 4초 ADE가 처음 보는 navtest보다 약 8배 작았다.

표 6. navtrain 내부 검증과 처음 보는 navtest의 차이

모델	navtrain head-val PDMS / ADE4	navtest-dev PDMS / ADE4
B10	93.12 / 0.076 m	90.93 / 0.598 m
B02-opt	92.75 / 0.077 m	90.88 / 0.599 m
v1 SF3-terminal (초기 head)	86.72 / 0.317 m	85.59 / 0.674 m

같은 이유로 navtrain feature에는 미래 궤적이 사실상 기억되어 있었다. 미래 궤적을 256개 anchor로 분류하는 head를 학습하자 학습 정확도는 100%에 이르렀지만 검증 정확도는 18.8%에 그쳤다. 이후 모든 결론은 처음 보는 navtest 장면을 기준으로 냈다. 고정된 world-model 표현을 평가할 때는 backbone의 학습 데이터와 평가 장면의 중복을 먼저 통제해야 한다는 교훈이다.

4.2 NAVSIM: 저-step 기준과 B* 선정 (dev)

navtest-dev에서 B10 대비 PDMS 차이는 B05-opt –0.03, B03-opt –0.04, B02-opt –0.05로 모두 품질 기준을 통과했다. B01-opt는 –1.19점에 DAC가 1.11점 악화되어 탈락했다. 따라서 NAVSIM의 B*는 **B02-opt**이며, 같은 세션에서 측정한 지연은 B10의 절반 수준이었다(B10보다 1.9배 빠름). 기존 DriveWAM만으로도 action step을 2까지 줄일 수 있다는 뜻이고, 제안 모델은 이보다 더 빨라야 의미가 있다.

4.3 표현 시점: first ≫ current, first ≈ terminal (두 벤치마크 공통)

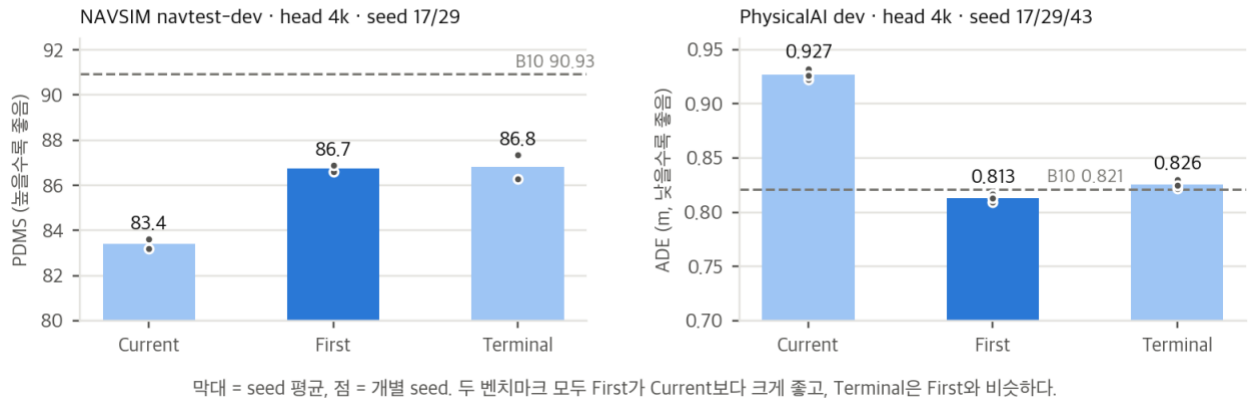


그림 7. 같은 SF3 head를 세 시점의 표현으로 각각 학습한 결과 (4k head). 왼쪽은 높을수록, 오른쪽은 낮을수록 좋다.

NAVSIM navtest-dev에서 first와 current의 PDMS 차이는 두 seed에서 +3.40, +3.28점이었고 신뢰구간은 0을 포함하지 않았다. 반면 first와 terminal의 차이는 seed 17에서 –0.74 [–2.11, +0.41], seed 29에서 +0.61 [+0.06, +1.16]으로 seed에 따라 방향이 달랐다. PhysicalAI dev에서도 같은 패턴이 나타났다. current는 first보다 ADE가

0.106~0.123 m, FDE가 0.41~0.46 m 나빴고(세 seed 모두 CI > 0), terminal은 ADE 차이 +0.008~+0.014 m로 신뢰구간이 0을 포함했다.

이 결과는 궤적을 읽는 데 필요한 정보가 미래 영상을 완성한 뒤에만 생기는 것이 아니라, backbone이 noise 상태의 미래 latent를 처음 처리하는 순간 이미 상당 부분 형성된다는 뜻이다. 나머지 denoising은 readout에 추가 정보를 거의 주지 않았다. 이것이 첫 pass에서 조기 종료할 수 있는 근거다.

4.4 head 학습 데이터 규모

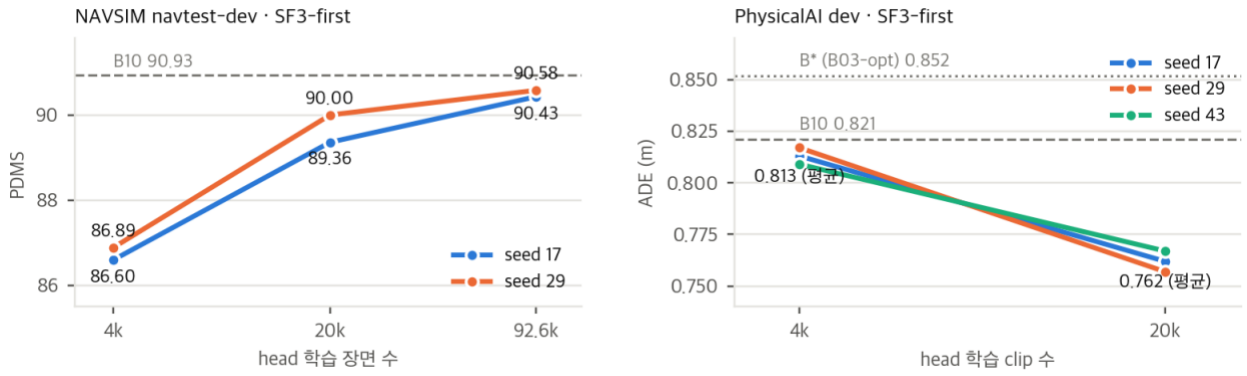


그림 8. head 학습 데이터 규모에 따른 SF3-first의 품질 (dev). 점선은 같은 세트의 B10.

표 7. NAVSIM navtest-dev에서의 데이터 규모 곡선 (SF3-first)

head 학습 데이터	PDMS (seed 17 / 29)	B10 대비 [95% CI]	DAC 악화	ADE4
4k	86.60 / 86.89	–4.3 / –4.0	–4.0	0.68 m
20k	89.36 / 90.00	–1.57 / –0.94	–1.51 / –1.05	0.624 m
92.6k (head-train 전체)	90.43 / 90.58	–0.51 [–1.12, +0.14] / –0.36 [–0.95, +0.28]	–0.52 / –0.40	0.61 m
B10	90.93	—	—	0.598 m

같은 구조에서 데이터만 늘렸을 때 B10과의 격차가 –4.2 → –1.3 → –0.4점으로 줄었다. PhysicalAI에서도 4k로 학습한 모델은 평균 ADE/FDE가 B10과 같았지만 FDE > 2 m 비율이 2.3~3.7%p 높아 기준(+1%p)에 미달했고, 20k에서는 ADE 0.762 m, FDE 2.299 m, FDE > 2 m 41.6%로 모든 기준을 통과했다. 표현에 action 정보가 들어 있다는 것과 작은 head가 그것을 해독할 수 있다는 것은 다른 문제이며, 후자에는 충분한 action 라벨이 필요하다. 초기 4k 결과만 보고 "world 표현에는 action 정보가 없다"고 결론 내렸다면 틀렸을 것이다.

4.5 깊이(H2): 방향은 일관, 확정은 보류

세 깊이를 쓰는 SF3와 파라미터 수를 맞춘 단일 깊이 head(SF1-wide)를 비교하면 SF3가 유리한 방향은 두 seed에서 일관했지만, 통계적으로 뚜렷한 차이는 한 seed에서만 나타났다. SF3의 성능이 좋다는 사실과 "여러 깊이가 필수"라는 주장은 다른 문제다. 두 head는 pooling 방식과 융합 구조도 다르므로 깊이 효과만 분리하려면 통제된 ablation이 더 필요하다. 현재 결과에서는 H2를 지지하는 신호는 있지만 확정하지 않는다.

4.6 NAVSIM 최종 평가 (잠금 navtest-final, 1회)

제안 모델은 head-train 전체(92.6k)로 학습한 SF3-first(seed 17, 29)다. 이 실행은 dev에서 한 장면 차이로 기준을 놓친 상태에서 수행한 기록된 예외다.

표 8. NAVSIM navtest-final 결과 (8,896장면, 102 로그)

모델	PDMS	NC	DAC	EP	ADE4	policy p50
B10	89.67	98.12	97.97	83.83	0.564 m	797.9 ms
B* = B02-opt	89.65	98.15	97.93	83.81	0.564 m	420.7 ms
제안 모델 seed 17	89.25	98.27	97.46	83.36	0.579 m	189.0 ms (B*보다 2.2배 빠름)
제안 모델 seed 29	89.49	98.30	97.62	83.52	0.577 m	189.0 ms (B*보다 2.2배 빠름)

B10 대비 PDMS 차이는 seed 17이 –0.42 [–0.76, –0.08], seed 29가 –0.17 [–0.56, +0.27]로 작았고, 충돌 지표 NC는 오히려 +0.15, +0.17 좋아졌다. 손실은 주행 가능 영역 준수(DAC)에 집중됐다. **사전등록 판정은 근소 미통과다.** seed 29는 모든 조건을 통과했지만, seed 17의 DAC 평균 악화가 0.51점으로 기준 0.5점을 넘었다. 위반 장면 수로는 B10 181개 대 제안 모델 226개(+45)이고 허용치가 44.48개였으므로 한 장면 차이다. 따라서 NAVSIM 결과는 "B10·B*에 가까운 계획 품질을 유지하면서 지연을 420.7 ms에서 189.0 ms로 줄였으나(B*보다 2.2배 빠름), DAC 기준을 한 seed에서 한 장면 차이로 넘지 못했다"로 정리한다.

4.7 PhysicalAI-AV 최종 평가 (공식 test 1k, 1회)

dev(1,000 clip, 614 chunk)에서 B*를 먼저 정했다. B05-opt(ΔADE +0.004)와 B03-opt(+0.031)는 자격 기준을 통과했고 B02-opt(+0.079, CI 상한 +0.089)는 탈락해 **B* = B03-opt**다. NAVSIM에서는 2 step까지 품질이 유지된 것과 달리 PhysicalAI에서는 3 step이 한계였다. head 학습 규모는 사전등록 규칙(seed 평균 dev ADE가 낮은 쪽)에 따라 20k를 택했다.

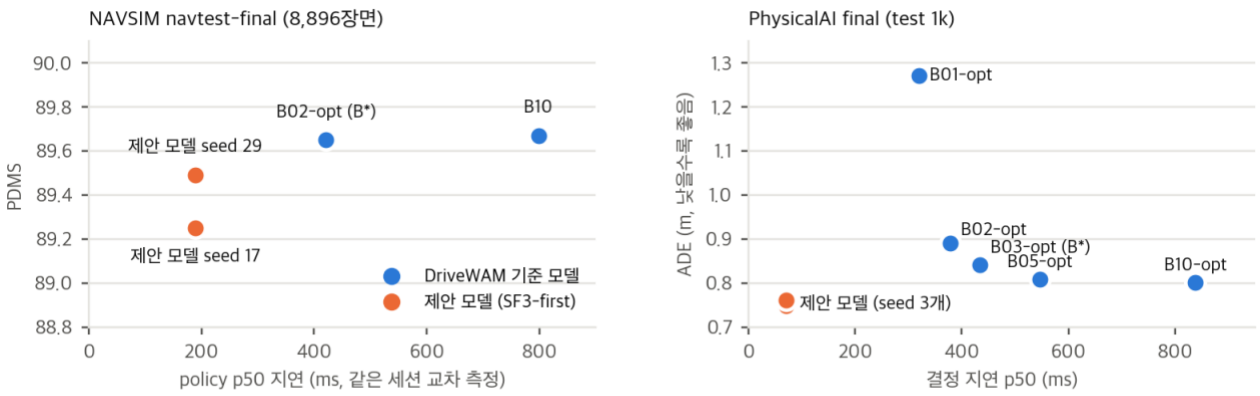
표 9. PhysicalAI final 결과 (test 1k, 480 NVIDIA chunk; 제안 모델 = 20k SF3-first)

모델	ADE (m)	FDE (m)	FDE > 2 m	ΔADE vs B10 [95% CI]	ΔFDE vs B10 [95% CI]	결정 p50
B10-opt (논문 0.83 / 2.47)	0.802	2.384	43.9%	—	—	837.8 ms
B* = B03-opt	0.842	2.362	42.5%	+0.040	–0.022	434.0 ms
제안 모델 seed 17	0.750	2.244	41.4%	–0.051 [–0.070, –0.035]	–0.140 [–0.198, –0.086]	70.9 ms
제안 모델 seed 29	0.748	2.244	41.0%	–0.054 [–0.072, –0.036]	–0.141 [–0.196, –0.088]	70.9 ms
제안 모델 seed 43	0.762	2.287	42.1%	–0.040 [–0.057, –0.023]	–0.097 [–0.152, –0.045]	70.9 ms

B* 행의 Δ는 B10 대비 평균 차이만 표기했다. 제안 모델의 B* 대비 차이는 ΔADE –0.080 ~ –0.094 m, ΔFDE –0.075 ~ –0.119 m(모두 CI < 0)다.

판정은 통과다. 세 seed 모두 사전등록한 7개 기준(B10·B* 대비 ADE/FDE, FDE > 2 m 비율, 속도)을 만족했다. 다만 제안 모델의 ADE/FDE가 B10보다 낮은 것을 정확도 우위로 읽지는 않는다. 이유는 4.9절의 결정성 대조에서 설명한다.

4.8 속도: 품질–지연 비교



왼쪽 아래(빠르고 정확)일수록 좋다. 제안 모델은 두 벤치마크 모두 B*보다 왼쪽에 있다.

그림 9. 최종 세트에서의 품질과 지연. 제안 모델은 두 벤치마크 모두 B*보다 빠르면서 품질이 그 근처에 있다.

표 10. 최종 지연 비교 (같은 세션 교차 3회 측정)

벤치마크	B*	B* 지연 (p50 / p95)	제안 모델 지연 (p50 / p95)	B* 대비	B10 대비
NAVSIM (final)	B02-opt	420.7 / 424.8 ms	189.0 / 192.2 ms	2.2배 빠름 (55% 감소)	4.2배 빠름 (76% 감소)
PhysicalAI (final)	B03-opt	434.0 / 502.9 ms	70.9 / 84.2 ms	6.1배 빠름 (84% 감소)	11.8배 빠름 (92% 감소)

B*는 action step을 줄이는 방법의 한계치다. 제안 모델은 거기서 더 나아가 남은 미래 denoising과 action 생성 전체를 생략하므로 추가 절감이 생긴다. PhysicalAI에서 실제 기록 등록까지 포함한 chunk당 처리 시간도 제안 모델 189.9 ms 대 B* 552.5 ms로, 모델 내부가 아니라 파이프라인 수준에서도 차이가 유지됐다.

4.9 PhysicalAI 추가 분석: 기억·언어·결정성

연속 주행 설정에서는 상상 외에 기억과 언어 안내가 readout에 얼마나 기여하는지도 진단했다(dev, seed 3개, first 대비).

표 11. PhysicalAI dev 진단 (4k head)

설정	ADE (m)	ΔADE vs first	ΔFDE vs first	해석
first (기준)	0.809 – 0.817	—	—	
current (상상 없음)	0.922 – 0.932	+0.106 ~ +0.123	+0.41 ~ +0.46	상상이 핵심
terminal (상상 3단계)	0.822 – 0.830	+0.008 ~ +0.014 (CI 0 포함)	+0.02 ~ +0.04	1 pass로 충분
과거 chunk 기억 제거	0.837 – 0.846	+0.020 ~ +0.037	+0.06 ~ +0.14	기억이 도움
VLM 안내 문장 제거	0.851 – 0.854	+0.034 ~ +0.045	+0.14 ~ +0.19	언어가 도움
원래 좌표계 사양 (회전 없음)	1.070 – 1.085	+0.260 ~ +0.269	+0.38 ~ +0.41	좌표계 처리가 결정적

기억과 언어는 모두 readout에 쓰이지만, 효과는 상상 유무의 차이(0.11 m)보다 작다. 안내 문장을 빼도 first는 current보다 0.07 m 이상 좋았으므로, first의 이점을 "다음 chunk의 안내 문장" 같은 교란으로 설명할 수는 없다.

결정성 대조. B10은 noise에서 출발하는 확률적 생성기이고 제안 모델은 같은 입력에 같은 궤적을 내는 결정적 회귀기다. 공식 protocol은 모든 clip에 같은 noise seed(0)를 쓰므로 seed가 점수 전체를 한 방향으로 움직인다. B10을 seed 0~3으로 돌리면 dev ADE가 0.821~0.930으로 크게 달랐고, 공식 seed 0이 가장 유리했다.

표 12. B10의 noise seed 민감도와 여러 표본 평균 (dev)

B10 (dev)	ADE (m)	FDE (m)
seed 0 (공식) / 1 / 2 / 3	0.821 / 0.869 / 0.882 / 0.930	2.452 / 2.453 / 2.542 / 2.647
2 / 3 / 4개 표본 평균	0.781 / 0.765 / 0.777	2.250 / 2.179 / 2.200
제안 모델 20k (seed 3개)	0.757 – 0.767	2.284 – 2.317

B10 네 표본의 평균과 비교하면 제안 모델의 ADE는 대등하고(ΔADE –0.011 ~ –0.021) FDE는 오히려 0.084~0.117 m 나쁘다(CI > 0). 따라서 공식 B10 대비 정확도 우위의 상당 부분은 결정성 효과다. 제안 모델의 기여는 "확률적 B10이 내는 궤적 분포의 평균 수준 예측을 한 번의 forward로, B*보다 6배 빠르게 낸다"는 데 있다.

5. 해석, 한계, 다음 단계

5.1 가설별 결론

표 13. 가설별 결론

가설	결론	근거
H1 표현의 충분성	대체로 지지	NAVSIM –0.17 ~ –0.42 PDMS (DAC 기준은 근소 미통과), PhysicalAI 기준 통과
H2 여러 깊이의 가치	보류	SF3가 유리한 방향은 일관하나 한 seed에서만 유의. 통제된 ablation 필요
H3 실용 효율	강하게 지지	B*보다 2.2배(NAVSIM), 6.1배(PhysicalAI) 빠름. 단계 수 감소보다 큰 절감
H4 표현 시점	first의 가치 확인	first ≫ current, first ≈ terminal 패턴이 두 벤치마크에서 일치

가장 강하게 지지되는 결론은 두 가지다. 첫째, 미래를 noise에서 한 번 상상하는 계산이 action 정보를 만들어 내고 그 뒤의 refinement는 readout에 거의 기여하지 않는다. 둘째, 그 시점의 표현을 작은 head로 읽으면 반복 action 생성을 없앨 수 있어 단계 수 감소보다 큰 지연 절감이 생긴다. 즉 DriveWAM에서는 좋은 미래 영상을 만드는 데 필요한 계산량과 좋은 action을 읽는 데 필요한 계산량이 같지 않다.

5.2 두 벤치마크의 판정이 갈린 이유

시점과 규모의 경향은 두 벤치마크에서 같았지만 최종 판정은 NAVSIM 근소 미통과, PhysicalAI 통과로 달랐다. 이것을 "PhysicalAI에서는 방법이 잘 되고 NAVSIM에서는 안 된다"로 읽기보다 다음 차이를 함께 봐야 한다.

- **지표.** NAVSIM의 PDMS는 주행 가능 영역 같은 규칙 위반에 민감해 작은 위치 차이가 큰 점수 차이가 될 수 있다. PhysicalAI의 ADE/FDE는 평균 거리이므로 결정적(평균) 예측에 유리하다.
- **설정.** PhysicalAI는 실제 기록이 계속 등록되는 연속 설정이라 제안 모델이 기억과 안내 문장의 정보를 추가로 쓸 수 있었다.

- **남은 약점의 성격.** NAVSIM의 DAC, PhysicalAI의 FDE 모두 여러 가능한 미래(예: 갈림길)를 하나의 평균 궤적으로 뭉개는 결정적 회귀의 문제와 관련 있을 수 있다. 이 가설은 아직 직접 검증하지 않았다.

5.3 한계

- world model은 DriveWAM 1종이고, 평가는 open-loop·비반응 채점이다. 폐루프 안전성은 검증하지 않았다.
- backbone은 이미 영상과 주행 action supervision으로 학습된 checkpoint이고 head도 궤적 라벨로 학습했다. action supervision 없이 능력이 생겼다는 연구가 아니다.
- 결정적 head와 확률적 생성기의 비교는 평균 오차 지표에서 제안 모델에게 유리하다. 다봉 분포나 안전 지표에서의 비교가 필요하다.
- PhysicalAI의 dev는 무작위 val(해시 순위)이고 공식 test는 VLM으로 선별된 장면이다. 다만 B10 ADE가 dev 0.821, final 0.802로 비슷했다. 좌표계 회전은 결과 전에 정했지만 결과를 크게 좌우했다(없으면 ADE +0.26 m).
- action 생성기의 1단계 증류 같은 다른 가속 방법과는 비교하지 않았다. 지연은 A5000 1장, 공유 호스트 기준의 비율이다.

5.4 다음 단계

- **추가 실험 후보.** ① 깊이 효과를 분리하는 통제된 ablation(H2), ② DAC 손실의 원인 분석과 다봉 분포를 다루는 출력(anchor·mixture), ③ 1단계 증류 기준과의 비교, ④ 다른 WAM이나 폐루프 평가로의 확장.

부록. 근거 파일 위치 (research_plan/ 기준)

내용	위치
NAVSIM navtest-dev (B*, 시점, 깊이, 지연)	evidence/navtest_dev_2026-10-02/
NAVSIM 규모 곡선 (20k, 92.6k)	evidence/round3_2026-10-03/, evidence/round4_2026-10-03/
NAVSIM final	evidence/navtest_final_2026-10-03/
PhysicalAI 전체 (parity, dev/final 분석, 진단, 결정성 대조)	evidence/physicalai_2026-10-04/ (reports/*.json, tables.md)
사전등록 문서	사전등록_navtest-dev_2026-10-02.md, 사전등록_PhysicalAI_2026-10-04.md
코드	world_feature_tap.py, lightweight_heads.py, heads_v3.py, pai_heads.py, pai_common.py, remote_*.py
통합 기록	통합_연구기록.md, README.md, PhysicalAI_결과보고서.md